

Proceso de carga de datos de competencia — Benchmark Leuk

Cómo se suben **precios**, **productos nuevos** y **competidores nuevos** al benchmark.

Documento técnico interno. **No** va a app/ (repo público): referencia rutas secretas

(service_role key, DB canónica).

0. Distinción clave: precios != productos/competidores

Hay **dos caminos** completamente distintos según qué querés hacer:

Querés...	Cómo	Toca código/pipeline
Actualizar precios de algo que ya existe	Desde la app (botón Precios)	No — override en tabla precios de Supabase
Sumar productos nuevos	Editar Excel fuente + re-correr el pipeline Python	Sí
Sumar un competidor nuevo	Editar Excel fuente + editar <code>paths.py/consolidate.py</code> + pipeline	Sí

- El botón **Precios** de la app matchea un Excel a las **entidades ya existentes** (por SKU Leuk o por `fs slug/nombre de competencia`) y guarda overrides en la tabla **precios**. **No crea productos ni competidores**.
- Sumar productos/competidores **no tiene UI todavía**: es un proceso de analista/dev sobre `~/leuk-benchmark/pipeline/`.

Todo lo que sigue es el segundo camino.

1. Dónde viven las fuentes (canónicas)

Definido en `pipeline/paths.py`. Por cada competidor (`Vonderk`, `Artelum`, `World Leds Go`)

hay **cuatro insumos**, todos bajo `| Comercial/.../Bases de datos/`:

Insumo	Archivo	Hoja	Contenido
Ficha técnica	Fichas técnicas/<Marca>_Fichas_Tecnicas_N# #.xlsx	Catálogo completo	~16 specs: potencia, lúmenes, eficiencia, IP, Fijación/montaje, medidas, Precio ...
Etiquetas de forma	Imágenes/imagenes_<marca>/<Marca>_Etiquetas_Forma.xlsx	Etiquetas_Forma	vocabulario visual controlado (tipo_forma, tipo_montaje, color, terminación, estilo, proporción)
Índice de imágenes	Imágenes/imagenes_<marca>/indice.csv	—	<code>producto -> urls</code> (varias URLs separadas por <code>\</code>)
Imágenes físicas	<code>~/imagenes_<marca>/</code>	—	los archivos para generar los embeddings visuales

Además, dos datos vienen de la **DB canónica** `~/leuk_pricing/pricing.db`:

- `TC_BLUE` (tabla `config`) — tipo de cambio ARS->USD (hoy ~ 1435).
- Descuentos** (tabla `descuentos_competidores`) — `marca -> desc_total`, para el precio neto.

El **pegamento** que une los tres "mundos de nombres" (fichas, etiquetas, imágenes) es una

función `slug()` normalizada. Ojo con la granularidad:

- las **fichas** están a nivel **familia** (`VK-VIKER`),
- las **etiquetas/embeddings** a nivel **variante** (`vk-viker-s`, `vk-viker-duo`).

`build_universe()` reconcilia por prefijo de slug y por token de nombre.

2. Sumar un PRODUCTO nuevo (a un competidor existente)

1. **Ficha** -> agregar la fila en `<Marca>_Fichas_Tecnicas_N##.xlsx`, hoja `Catálogo completo`.

Campos que usa el motor (`FICHA_CAMPOS` en `consolidate.py`): Fuente de luz, Potencia (W), Lúmenes del sistema, Eficiencia (lm/W), Tensión (V), Ángulo de haz, CRI, Temp. de color (K), UGR, IP, Fijación/montaje, Medidas, Movimiento, Color del artefacto, Control, Garantía.

Precio:

- **Vonderk** -> columna `Precio` (ya en USD).
 - **Artelum / WLG** -> columna `Precio ($)` en ARS (el pipeline lo divide por `TC_BLUE`).
2. **Imagen** -> poner el/los archivo(s) en `~/imagenes_<marca>/` y sumar la fila `producto_urls` en `indice.csv`. El ranking `_rank_img_url()` elige la mejor foto de producto: prioriza `portada/carrusel-producto`, penaliza `ambient/render`, y descarta `logo/cota/diagrama`.

3. **Etiqueta de forma** -> agregar la fila en `<Marca>_Etiquetas_Forma.xlsx` (o generarla con el tagger de visión; para WLG existe `wlg_tag.py`).

4. **Re-embeber lo visual** (paso pesado, OpenCLIP ViT-B-32 -> vector 512 dim normalizado). Según el competidor:

- **WLG** -> `python3 wlg_build.py` -> regenera `wlg_vecs.npz` + `wlg_families.json`.
- **Artelum** -> `artelum_build.py` / `artelum_bundle.py` -> `artelum_vecs.npz`.
- **Vonderk** -> vía el índice **FAISS** (`pipeline/faiss/catalog.faiss`), que después `img_comp_vecs.py` promedia por slug -> `comp_slug_vecs.npz`.

5. **Consolidar y subir** (ver §5).

Nota: la onboarding visual **no es uniforme** entre marcas. Vonderk entró por el índice

FAISS; WLG y Artelum por scripts de build dedicados con su propio `<marca>_vecs.npz`.

`cargar_artefactos_leuk()` mergea todos esos `.npz` en un solo `comp_vecs`.

3. Sumar un COMPETIDOR nuevo (marca)

Además de todo lo del §2, hay que tocar código:

1. `consolidate.py` -> agregar la marca a `MARCAS = ["Vonderk", "Artelum", "World Leds Go"]`.
2. `paths.py` -> sumar la entrada en los 4 diccionarios: `FICHAS`, `ETIQUETAS`, `INDICES`, `IMG_COMP_SRC`.
3. Crear los **4 insumos** (ficha Excel `Catálogo completo`, etiquetas Excel `Etiquetas_Forma`, `indice.csv`, carpeta de imágenes).
4. Escribir/adaptar un **script de build** de embeddings (base: `wlg_build.py`): agrupa imágenes por sub-producto, descarta diagramas/ambientadas con heurística de bordes, embebe con OpenCLIP, guarda `<marca>_vecs.npz` + `<marca>_families.json` + una foto representativa en

app/img/<marca>/. Sumar ese .npz al merge de cargar_artefactos_leuk().

5. **Precio/descuento** -> cargar el descuento de la marca en descuentos_competidores (DB). Para modelos sin ficha, usar overrides curados (<marca>_precio_manual.json, o el dict

ART_PRECIO_MANUAL para casos puntuales de Artelum).

6. **La app se entera sola**: el front lee MARCAS = DATA.meta.competidores del JSON, así que la marca nueva aparece en filtros, catálogo y "mejor por competidor" **sin tocar** app.js.

4. Qué hace la consolidación (consolidate.py -> benchmark_data.json)

1. build_universe() arma el **universo de competencia** por slug reconciliado. Cada entidad junta las señales que existan: **etiqueta** (forma), **vis_vec** (imagen) y **variantes** de ficha

(specs + precio). Une los 3 mundos de nombres (etiquetas/embeddings vs fichas vs índice).

2. Para cada producto **Leuk** (ancla), evalúa contra **todas** las entidades de cada marca con

3 señales:

- **Técnica** — matching.py compara specs fotométricas, con *gating* por IP/montaje.
- **Etiquetación** — coincidencia de forma/estética (vocabulario controlado). El **color se ignora**: es variante (NG/BL/MD), no identidad.
- **Visual** — similitud coseno de los embeddings OpenCLIP.

3. Combina en un **veredicto** (Equivalente / Comparable parcial / Posible / No comparable)

con una **confianza** (alta = >=2 señales). Rankea por confianza -> nivel -> score -> desempate por ficha.

4. Calcula precios **netos** (lista × (1 - desc)) para Leuk y competidor, y la diferencia %.

5. Escribe por producto: mejor_por_marca, propuestas (confiables, <=20), posibles (1 señal, a revisar, <=15), similares (otros Leuk parecidos -> señal de recomendación), y un catalogo plano de toda la competencia (para el buscador de "sugerir a mano").

6. **Salidas**:

- data/benchmark_data.json — **canónico** (lo que sube a Supabase).
 - app/data.js — legacy (window.BENCHMARK = ...), ya **no** se usa.
 - meta: generado, competidores (= MARCAS), tc_blue.
-

5. Correr y publicar

```
cd ~/leuk-benchmark/pipeline
```

```
bash run_all.sh # 8 pasos: imágenes Leuk -> vecs competencia -> tags -> consolidate -> fichas
```

```
python3 subir_datos.py # sube benchmark_data.json al bucket privado 'datos' de Supabase (upsert)
```

- run_all.sh **no** corre subir_datos.py — es un paso aparte.
- subir_datos.py usa la **service_role key** de ~/.leuk_pricing/supabase_service.txt (secreta, nunca en el repo) y hace un POST ... x-upsert:true al Storage.

- La app baja ese JSON **tras el login** (fetchData()) con el JWT del usuario). **No** hay que republicar la app: los datos viven en Supabase, el código en GitHub Pages.

Dependencias del entorno (una vez)

```
pip install pandas openpyxl pydantic numpy pillow open_clip_torch faiss-cpu anthropic gdown
```

torch es pesado; faiss y torch se corren en procesos separados porque juntos segfaultean

en macOS (por eso `img_comp_vecs.py` está aislado del resto).

6. Capas de precio (resumen)

Un mismo producto puede tener precio de tres fuentes, en este orden de resolución:

1. **Ficha Excel** — **Precio** (Vonderk, USD) o **Precio** (\$) (Artelum/WLG, ARS->USD por `TC_BLUE`).
2. **Override manual del pipeline** — para entidades **sin ficha**: `<marca>_precio_manual.json`, `vonderk_precios_pdf.json`, o el dict `ART_PRECIO_MANUAL` en `consolidate.py`.
3. **Override runtime de la app** — tabla `precios` de Supabase, cargada con **Precios**; se aplica al mostrar (no toca el JSON base).

El **neto** siempre es `lista × (1 - descuento)`, con el descuento de `descuentos_competidores` (o el que se edite en vivo con **Descuentos**).

7. Estado actual y deuda técnica

- El proceso es **de analista/dev**, no self-service: editar Excels + correr Python con dependencias pesadas (OpenCLIP, FAISS, Claude visión) + subir.
- Lo más frágil/manual:
 1. **Onboarding visual no uniforme** entre marcas (FAISS vs scripts de build por marca).
 2. **Precios manuales curados** a mano para entidades sin ficha.
- **Próximos pasos naturales**:
 - Unificar los scripts de build por marca en **uno parametrizado** (`build_competidor.py <marca>`).
 - Automatizar la carga **desde la app** (subir ficha + imágenes de un producto y disparar la re-consolidación), para sacar el proceso del entorno local.

8. Mapa de archivos del pipeline (referencia rápida)

Archivo	Rol
<code>paths.py</code>	rutas de todas las fuentes y salidas
<code>consolidate.py</code>	motor de matching en vivo -> <code>benchmark_data.json</code> (MARCAS, <code>build_universe</code> , <code>evaluar</code> , <code>main</code>)
<code>matching.py</code>	señal técnica (specs + gating)
<code>wlg_build.py</code> / <code>artelum_build.py</code> / <code>artelum_bundle.py</code>	embeddings visuales de cada competidor -> <code><marca>_vecs.npz</code>
<code>wlg_tag.py</code>	etiquetado de forma de WLG
<code>img_comp_vecs.py</code>	reconstruye vectores de competencia desde FAISS -> <code>comp_slug_vecs.npz</code>
<code>img_leuk_*.py</code>	manifiesto, descarga, embeddings y etiquetado de imágenes Leuk
<code>fichas_build.py</code>	fichas técnicas -> <code>app/fichas-data.js</code>
<code>run_all.sh</code>	orquestra los 8 pasos
<code>subir_datos.py</code>	sube <code>benchmark_data.json</code> a Supabase Storage

Archivo	Rol
<marca>_precio_manual.json, vonderk_precios_pdf.json	overrides de precio para entidades sin ficha